

תרגיל בית 2 - מבוא למחשוב ענן

צוות - Zebra



פרח שנבחר: סחלב



 GreenZebra	קישור למחברת הגשה:
GitHub	קישור לתיקיית גיט:

שם	תפקיד	משימות שהוקצו	משימות שהושלמו
תומר לוי	מהנדס מערכת + UI	חלק שני שניידרמן, SUS ומדדים	הכל הושלם
דניל גסן	Backend developer	בניית אינדקס סעיף ב, מסך שאילתא למנוע חיפוש, דשבורד ויזואלי של מצב צמח	הכל הושלם
ירין חממי	QA	פיצ'ר, דגימת נתונים מחיישי IOT	הכל הושלם
טדי חדאד	Frontend developer	בניית אינדקס סעיף א, חלק שני קלוד	הכל הושלם
לאון סיגל	Product manager	חלק שני העלאת תמונות צמחים, ארכיטקטורה	הכל הושלם

Document Number	Article Name	Article Authors	URL link
Doc 1	Intelligent image analysis recognizes important orchid viral diseases	Cheng-Feng Tsai Chih-Hung Huang Fu-Hsing Wu Chuen-Horng Lin Chia-Hwa Lee Shyr-Shen Yu Yung-Kuan Chan Fuh-Jyh Jan	https://www.frontiersin.org/journals/plant-science/articles/10.3389/fpls.2022.1051348/full
Doc 2	A molecular perspective on orchid development	Khosro Balilashaki Shirin Moradi Maryam Vahedi Amir Ali Khoddamzadeh	https://www.tandfonline.com/doi/full/10.1080/14620316.2020.1727782
Doc 3	Orchid Diversity: Conservation and Utilization	S. P. Vij Promila Pathak	https://link.springer.com/article/10.1007/s40011-012-0107-0
Doc 4	Colour-scent associations in a tropical orchid: Three colours but two odours	Roxane Delle-Vedove Nicolas Juillet Jean -Marie Bessière Claude Grison Nicolas Barthes Thierry Paillet Laurent Dormont Bertrand Schatz	https://www.sciencedirect.com/science/article/pii/S0031942211000744
Doc 5	Monitoring and Control System Watering Orchid Plants Based on Weather Data	Rosih Ihwan Unit Kartini	https://journal.unesa.ac.id/index.php/inajet/article/view/33340

ב.בחרו מתוך המאמרים לפחות 20 מילים משמעותיות עבורן תבנו אינדקס.

orchid	term
Doc 1, Doc 2, Doc 3, Doc 4, Doc 5	DocIDs
disease	term
Doc 1, Doc 3, Doc 5	DocIDs
detection	term
Doc 1, Doc 2, Doc 4	DocIDs
development	term
Doc 1, Doc 2, Doc 3, Doc 4, Doc 5	DocIDs
temperature	term
Doc 1, Doc 3, Doc 4, Doc 5	DocIDs
humidity	term
Doc 1, Doc 5	DocIDs
sensor	term
Doc 1, Doc 5	DocIDs
moisture	term
Doc 5	DocIDs
watering	term
Doc 5	DocIDs
micropropagation	term
Doc 2, Doc 3	DocIDs
conservation	term
Doc 2, Doc 3, Doc 4	DocIDs
germination	term
Doc 2, Doc 3	DocIDs
diversity	term
Doc 1, Doc 2, Doc 3, Doc 4	DocIDs

phalaenopsis	term
Doc 1, Doc 2, Doc 3	DocIDs
leaf	term
Doc 1, Doc 2, Doc 3, Doc 4, Doc 5	DocIDs
growth	term
Doc 1, Doc 2, Doc 3, Doc 5	DocIDs
medium	term
Doc 3, Doc 4, Doc 5	DocIDs
identification	term
Doc 1	DocIDs
analysis	term
Doc 1	DocIDs
model	term
Doc 1	DocIDs
accuracy	term
Doc 1, Doc 5	DocIDs
temperature	term
Doc 1, Doc 3, Doc 4, Doc 5	DocIDs
monitor	term
Doc 5	DocIDs
control	term
Doc 5	DocIDs
system	term
Doc 1, Doc 5	DocIDs
data	term
Doc 5	DocIDs

environment	term
Doc 5	DocIDs
flower	term
Doc 2, Doc 3, Doc 4	DocIDs
root	term
Doc 2, Doc 3	DocIDs
seed	term
Doc 2, Doc 3	DocIDs
tissue	term
Doc 2, Doc 3	DocIDs
culture	term
Doc 2, Doc 3	DocIDs

בחרנו ב stop-words הבאים:

מילים אלה הם – מילות יחס, מילות חיבור, מילות שאלה, פעלי עזר אשר אינם תורמים למציאת מידע, ולרוב הם רק מחברים בין מידע שנחפש בטקסטים, אם אנחנו נכניס אותם לאינדקס זה ימשוך את כל המסמכים ומנגנון חיפוש יהיה חסר שימוש.

```
STOP_WORDS = {
```

```
"a", "an", "and", "are", "as", "at", "be", "by", "for", "from", "in", "is", "it", "of",
"on", "the", "to", "with",
```

```
"based", "using", "what", "how", "who", "which", "why", "when", "where", "this", "th
at", "these", "those",
```

```
"they", "them", "their", "we", "our", "can", "will", "do", "has", "have", "would", "s
hould", "could",
```

```
}
```

בחירת *stem/lemmatization*:

בפרויקט שלנו בחרנו ב *lemmatization*. מסיבה כי שיטה זו ממירה מילים לצורת בסיס ותקנית, כאשר *stemming* חותך סיומות של מילים. כך אם נבחר בשיטה זו, זה יכול לייצר לנו מילים שהם חסרי משמעות כמו שזה יחתוך *leaves* להיות *leav*.

כאשר שיטה *lemmatization* תעביר אותה מילה *leaves* לצורה *leaf* שזה ייתן לנו יותר מידע נגיש ומועיל בחיפוש. בפרויקט שלנו פחות קריטי לנו מהירות התגובה, לעומת הדיוק התגובה שנקבל. אם נקבל מידע ששגוי או לא מועיל וטיפול אחר צמח, זה יוביל משתמש לתוצאות מבלבלות.

שמונת כללי הזהב של שניידרמן

1. **עקביות:** המערכת שומרת על שפה עיצובית אחידה. לאורך כל המסכים נעשה שימוש באותו גופן, אותה פלטת צבעים מבוססת ירוק לכפתורים מרכזיים ומדדי הצלחה, ואייקונים קבועים המייצגים את אותם מדדים גם במסך חיישני ה-*IoT* וגם בדשבורד המרכזי.
2. **קיצורי דרך:** משתמשים יכולים להעלות תמונה למערכת בשתי דרכים: לחיצה רגילה על אזור ה-*Drop Zone* או בשימוש *Drag and Drop*. בנוסף, במנוע ה-*RAG*, ניתן להפעיל את החיפוש לא רק בלחיצה על הכפתור אלא גם בקיצור המקלדת של לחיצה על מקש ה-*Enter*.
3. **משוב אינפורמטיבי:** המערכת מצטיינת במתן משוב חזותי. בעת לחיצה על "Fetch" *readings*, מופיע מיד אייקון טעינה יחד עם הטקסט "Fetching...". לאחר סיום העלאת תמונת סחלב, מופיעה תצוגה מקדימה מוקטנת של התמונה עם שם הקובץ שמאשרת את קליטתה. גם בדשבורד המרכזי ניתנים משובים ברורים בצבעים למשל, רקע ירוק "OK" או רקע צהוב "Out of range".
4. **סגירת מעגל:** תהליך החיפוש במנוע ה-*RAG* מסתיים בצורה ברורה כאשר המערכת מציגה למשתמש פסקה מסכמת, יחד עם תגיות לחיצות של הקישורים למאמרים. פעולה זו נותנת תחושה של סיום מוצלח של שאילתת החיפוש.
5. **מניעת שגיאות:** מסך הסורק תוכנן למנוע שגיאות. הוא מסנן אוטומטית קבצים שאינם תמונות ומציג שגיאה ברורה ("Only image files are allowed"), וכן מגביל את הגודל ל-10MB ומעלה הודעת שגיאה בהתאם. בנוסף, לא ניתן לבחור מספר חיישנים במקביל באופן שגוי במסך ה-*IoT*, בחירת חיישן מבטלת אוטומטית את הבחירה בחיישן הקודם.

6. **ביטול פעולה בקלות:** במסך הסורק סחלבים, לאחר שהמשתמש בחר תמונה, מופיע באופן בולט כפתור "Remove photo" יחד עם כפתור "X" קטן על גבי התצוגה המקדימה עצמה, המאפשרים לבטל את ההעלאה באופן מיידי ולחזור למצב ההתחלתי. גם במנוע ה-RAG ישנו כפתור "X" ברור למחיקת השאילתה הקיימת והתוצאות שלה.
7. **שליטה פנימית בידי המשתמש:** המשתמש שולט על כמות המידע שיוצג לו במסך ה-IoT על ידי סליידר שמאפשר לבחור את מספר הדגימות המדויק שנע בין 1 ל-50. כמו כן, הדשבורד לא כופה עדכון נתונים אוטומטי ומבלבל, אלא מאפשר למשתמש לשלוט מתי לרענן את הנתונים באמצעות לחיצה על כפתור "Refresh sensors".
8. **הפחתת עומס זיכרון:** הדשבורד המרכזי מחשב ומרכז עבור המשתמש את כל התובנות. המשתמש לא צריך לזכור מהם טווחי הטמפרטורה האידיאליים או לנתח את הנתונים המספריים, מכיוון שהמערכת מספקת "Health Score" באחוזים ומתרגמת את הנתונים להמלצות ברורות בשפה פשוטה, למשל: "Move the orchid away from cold windows".

התשובות של קלוד על המסכים:

שאלון SUS (כל פריט בסולם, 1-5 כאשר 1 = כלל לא, 5 = לחלוטין):

1. **הייתי רוצה להשתמש במערכת זו לעיתים תכופות** — 4 המערכת מציעה ניטור בזמן אמת שימושי למגדלי סחלבים.
2. **מצאתי כי המערכת מסובכת ללא סיבה** — 2 הממשק נראה נקי ואינטואיטיבי יחסית.
3. **חשבתי שהמערכת קלה לשימוש** — 4 הניווט בין שלושת הלשוניות (Scanner / IoT / Dashboard) פשוט וברור.
4. **אזדקק לתמיכת איש טכני כדי שאוכל להשתמש במערכת זו** — 2 הממשק ידידותי מספיק למשתמש רגיל.
5. **מצאתי כי הפונקציות השונות של המערכת היו מתואמות היטב** — 4 הסריקה, חיישני ה-IoT והדאשבורד משתלבים היטב זה בזה.
6. **חשבתי כי היה יותר מידי חוסר עקביות במערכת זו** — 2 עיצוב אחיד ועקבי בכל המסכים.
7. **לדעתי רוב האנשים יהיו מסוגלים ללמוד להשתמש במערכת זו בקלות** — 4 הממשק פשוט וברור מספיק.
8. **מצאתי כי המערכת מאוד מסורבלת לשימוש** — 1 המערכת נראית נוחה וזורמת.
9. **חשתי בטחון רב כאשר השתמשתי במערכת** — 4 ה-AI Care Advice וציוני הבריאות מעניקים תחושת שליטה ובטחון.
10. **עלי ללמוד הרבה דברים לפני שאוכל להשתמש במערכת זו** — 1 המערכת נראית accessible מיידית.

שאלון Engagement (כל פריט בסולם, 1-7 כאשר 1 = כלל לא, 7 = לאורך כל השימוש):

1. **הזמן "טס" במהלך ההתנסות** — 5
2. **ביצעתי את ההתנסות בהנאה רבה** — 5
3. **ההתנסות הייתה בעלת משמעות ומטרה** — 6 עבור מגדל סחלבים, הנתונים בעלי ערך ממשי.
4. **ההתנסות הייתה מאתגרת עבורי** — 2
5. **התמדתי בהתנסות גם כאשר הדברים לא התקדמו בצורה טובה** — 5

תשובות לשאלות הפתוחות:

האם לדעתך המערכת מותאמת לאוכלוסייה אליה היא מכוונת? כן — המערכת מכוונת למגדלי סחלבים מתחביב עד מקצועי. השפה פשוטה, הוויזואליות ברורה, ו-AI Advice מסביר המלצות בשפה נגישה. ייתכן שמשתמשים מבוגרים יתקשו מעט עם רכיב ה-IoT.

אלמנטים שאהבת במערכת:

- דאשבורד עם ציון בריאות מיידי וברור
- עצות ה-AI ספציפיות ומעשיות
- פלטת הצבעים הירוקה — עקבית ומתאימה לתחום

אלמנטים שיש לשפר במערכת:

- ציון בריאות 0% עם "Critical" — חסר הסבר כיצד לשפר
- אין פידבק ויזואלי ב-upload zone של הסורק מעבר לטקסט
- "Care Tracker" שמציג "...Loading" מפחית אמינות

טבלת התייחסות למשוב:

הערת משוב	האם התבצע שינוי באפליקציה בעקבות ההערה?	נימוק
ציון בריאות 0% עם "Critical": חסר הסבר כיצד לשפר	לא	כרטיסיות ה-AI Care Advice, הממוקמות ישירות מתחת לציון הבריאות בדשבורד, כבר מספקות את ההסברים המפורטים וההנחיות האופרטיביות כיצד לשפר את המצב (למשל "Provide", "Stop watering", "shade"), לכן אין צורך לשכפל את המידע.
אין פידבק ויזואלי ב-upload zone של הסורק מעבר לטקסט	כן	וידאנו וחידדנו את הפונקציה <code>addThumbs</code> כך שברגע שמועלית תמונה, היא מייצרת תצוגה מקדימה (Thumbnail) המחליפה את אזור הגרירה המקורי, כדי לספק אישור חזותי ברור ומיידי לקליטת התמונה.
"Care Tracker" שמציג "...Loading" מפחית אמינות	כן	הרכיב תוקן וחובר בהצלחה למסד הנתונים ולפונקציות העדכון בקולאב, כך שכעת הוא מציג את היסטוריית הטיפול האמיתית ואת רצף הימים במקום להישאר במצב טעינה אינסופי.

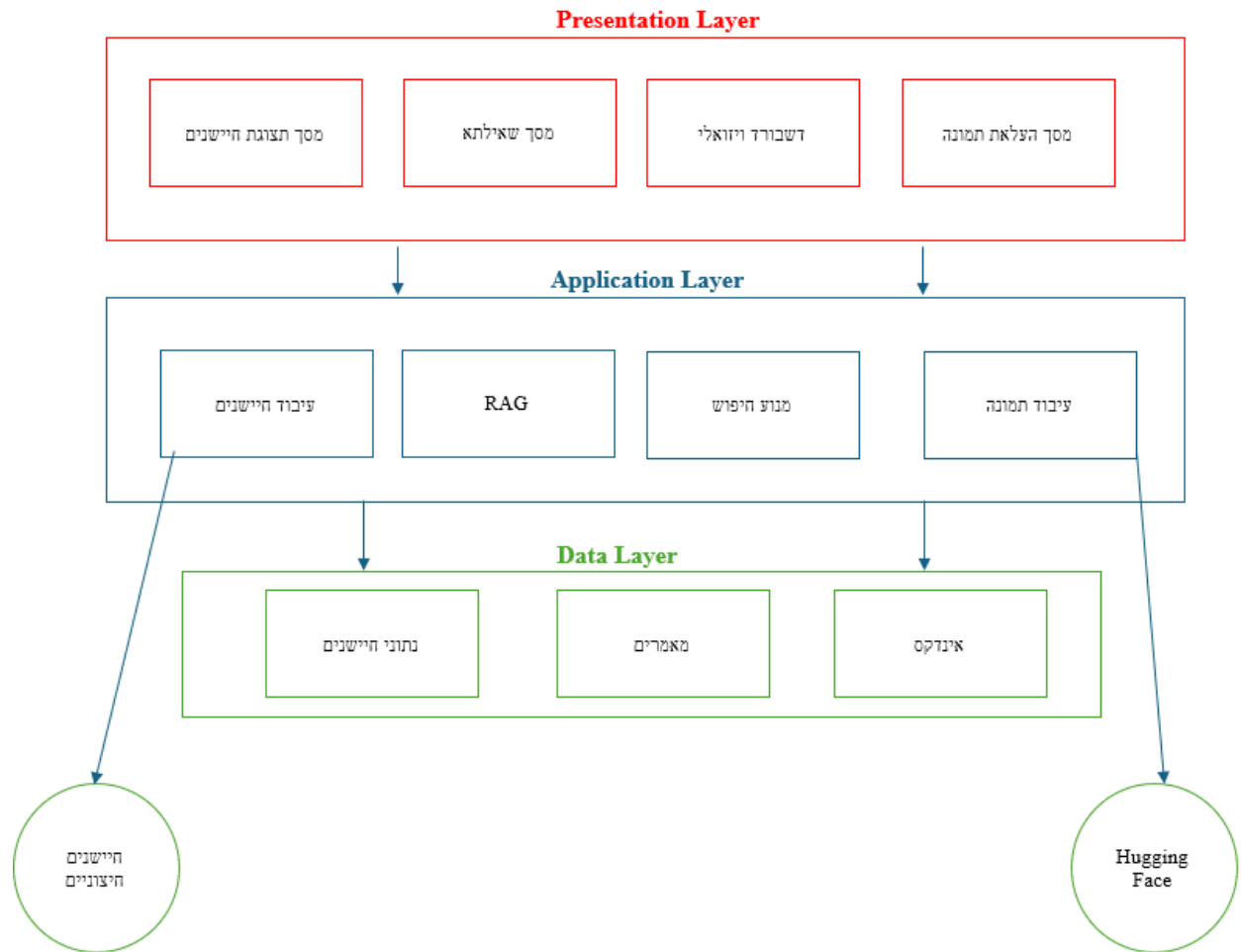
ציון ה-SUS:

- עבור שאלות אי-זוגיות: $(4-1) \times (4-1) \times (4-1) \times (4-1) \times (4-1) = 15$
- עבור שאלות זוגיות: $(5-1) + (5-1) + (5-2) + (5-2) + (5-2) = 17$
- נסכום את התוצאות ונכפול ב-2.5 ונקבל שציון ה-SUS הסופי של המערכת הוא 80.

מדדים להצלחת המערכת:

1. **אחוז הצלחת משימות במנוע החיפוש:** המדד בוחן כמה פעמים משתמש הזין שאילתה (למשל חיפוש על מחלות סחלבים) והקליק על אחת התוצאות מהאינדקס שהופקו על ידי ה-RAG. אחוז גבוה יעיד שהאינדקס שבנינו רלוונטי ומביא ערך אמיתי למשתמש.
2. **שיעור משתמשים פעילים יומי/חודשי:** מדד שבוחן כמה משתמשים נכנסים לדשבורד לפחות פעם אחת ביום כדי לבדוק את נתוני לחיישני ה-IoT של הצמח. מדד גבוה מעיד על רמת מעורבות טובה ועל כך שהמערכת מספקת תועלת מתמשכת.
3. **זמן תגובה של המערכת:** מדידת הזמן שלוקח לנתוני החיישנים (לחות/טמפרטורה) להתעדכן במסך הדשבורד מזמן טעינת העמוד, וגם הזמן שלוקח להחזיר תוצאות חיפוש. עמידה ביעד של מתחת ל-2 שניות לפעולה תעיד על ארכיטקטורה יעילה וחוויית משתמש זורמת.

תרשים ארכיטקטורה – 3 שכבות.



הארכיטקטורה בה בחרנו להשתמש הינה ארכיטקטורת "3 שכבות", אשר מחלקות ל3 שכבות שהן: שכבת התצוגה, שכבת הלוגיקה ושכבת הנתונים.

שכבת **התצוגה** מורכבת מ 4 מסכים: מסך העלאת תמונה, מסך שאילתא למנוע החיפוש, מסך דגימת נתונים מחיישני IoT ודשבורד ויזואלי של מצב הצמח.

שכבת **הלוגיקה** מורכבת מ: עיבוד תמונה, בניית אינדקס, מנוע חיפוש, RAG ועיבוד חיישנים. שכבת **הנתונים** מורכבת מ: אינדקס, מאמרים ונתוני החיישנים.

בנוסף לשכבות יש לנו גם שני רכיבים חיצוניים למערכת שנעבוד איתם: HF לצורך עיבוד תמונה וזיהוי מחלות, וחיישנים חיצוניים המספקים נתוני סביבה בזמן אמת.

רק נציין **שעיבוד תמונה + עיבוד חיישנים + נתוני חיישנים** זה דברים שאנחנו נוסף בעבודה מספר 3, לכן אין קטעי קוד לגבי החלקים הללו.

מנוע חיפוש

```
#Pulling from firebase
for word in query_words:
    #Check Firebase for the word
    doc_ref = db.collection('orchid_index').document(word).get()
    if doc_ref.exists:
        #If word is found in database, extract the links
        db_data = doc_ref.to_dict()
        retrieved_links.update(db_data["DocIDs"])

    for link in db_data["DocIDs"]:
        doc_num = int(link.split(':')[0].replace('Doc ', ''))
        retrieved_doc_ids.add(doc_num)
```

אינדקס

```
#Building index by lemmatization
lemmatizer = WordNetLemmatizer()

target_words = {
    "orchid", "phalaenopsis", "disease", "detection",
    "identification", "analysis", "model", "accuracy", "sensor",
    "temperature", "humidity", "moisture", "watering", "monitor",
    "control", "system", "data", "environment", "leaf", "flower",
    "root", "seed", "tissue", "culture", "growth", "development",
    "germination", "micropropagation", "propagation", "medium", "diversity", "conservation"
}

inverted_index = defaultdict(list)

for doc_id, doc_info in documents.items():
    if not doc_info.get("text"): continue

    text = doc_info["text"]
    words = text.lower().replace('.', '').replace(',', '').split()
    numbered_link = f"Doc {doc_id}: {doc_info['url']}"

    for word in words:
        if word not in stop_words:
            lemma = lemmatizer.lemmatize(word)
            if lemma in target_words:
                if numbered_link not in inverted_index[lemma]:
                    inverted_index[lemma].append(numbered_link)
```

RAG

```
#RAG Query
def display_rag_results_from_firebase(query, docs):
    print(f"RAG QUERY: '{query}'")
    print("-" * 70)

    query_words = [lemmatizer.lemmatize(w.lower()) for w in query.split() if w.lower() not in stop_words]

    retrieved_links = set()
    retrieved_doc_ids = set()

    if not retrieved_links:
        print("Response: No relevant information found in the attached articles.\n")
        return

    #Generation/Display Phase
    print("DASHBOARD RESPONSE: Based on the retrieved articles, here is the context for your query:\n")
    print("SOURCES FOUND:")
    for link in sorted(retrieved_links):
        print(f"Link: {link}")

    print("\nEXTRACTED TEXT:")
    for doc_id in sorted(retrieved_doc_ids):
        text = docs[doc_id]["text"]
        lower_text = text.lower()

        best_index = -1
        for word in query_words:
            idx = lower_text.find(word)
            if idx != -1:
                best_index = idx
                break

        if best_index != -1:
            start = max(0, best_index - 50)
            end = min(len(text), best_index + 300)
            snippet = text[start:end].replace("\n", ' ')
            print(f"Doc {doc_id} Relevant Snippet: ...{snippet}...\n")
        else:
            snippet = text[:300].replace("\n", ' ')
            print(f"Doc {doc_id} Snippet: {snippet}...\n")

    print("-" * 70)
```

מאמרים

```
#Document Download
document_sources = [
    {
        "original_url": "https://www.frontiersin.org/journals/plant-science/articles/10.3389/fpls.2022.1051348/full",
        "drive_download_url": "https://drive.google.com/uc?export=download&id=1mVxZKY9KzHYyImf_F1pVoox4EeUeX6n"
    },
    {
        "original_url": "https://www.tandfonline.com/doi/full/10.1080/14620316.2020.1727782",
        "drive_download_url": "https://drive.google.com/uc?export=download&id=1V5496mH6sD0t4gWkx38LnAnCh11f5"
    },
    {
        "original_url": "https://link.springer.com/article/10.1007/s40011-012-0107-0",
        "drive_download_url": "https://drive.google.com/uc?export=download&id=1vUmjW47V57VcPcfH6FatnIpQCzKik"
    },
    {
        "original_url": "https://www.sciencedirect.com/science/article/pii/S0031942211000744",
        "drive_download_url": "https://drive.google.com/uc?export=download&id=1mJnx-2_48s4g2ayQEY0wX3qHpaJ3eX"
    },
    {
        "original_url": "https://journal.unesa.ac.id/index.php/inajet/article/view/33348",
        "drive_download_url": "https://drive.google.com/uc?export=download&id=1lx0L7H6v8V3L08sA7F6xCqUigW_fom-"
    }
]

documents = {}
print("Downloading article texts from Google Drive...")

for doc_id, info in document_sources.items():
    try:
        response = requests.get(info["drive_download_url"])
        response.raise_for_status()
        documents[doc_id] = {
            "url": info["original_url"],
            "text": response.text
        }
    except Exception as e:
        print(f"ERROR: Failed to download Document {doc_id}.")

print("All documents downloaded successfully.\n" + "-" * 50)
```

מסכים

נמצא ב 7 cell. - כל המסכים בתא אחד!

```
0 #cell 7|

# =====
# The Green Zebra
# =====
import requests, json, firebase_admin
from datetime import datetime, timedelta, timezone
from IPython.display import HTML, display
from firebase_admin import firestore
from nltk.stem import WordNetLemmatizer
from google.colab import output as colab_output

# --- Shared configuration ---
BASE_URL = "https://server-cloud-v645.onrender.com"
IL_TZ = timezone(timedelta(hours=3))

THRESHOLDS = {
    "temperature": {"min": 15, "max": 30, "unit": "°C", "icon": "🌡️", "label": "Temperature"},
    "humidity": {"min": 40, "max": 70, "unit": "%", "icon": "💧", "label": "Air Humidity"},
    "soil": {"min": 30, "max": 70, "unit": "%", "icon": "🌱", "label": "Soil Moisture"},
}
```

הפיצ'ר שהוספנו למערכת:

הוספנו מערכת התראות חכמות המנתחת את הדגימות שנמשכו מחיישני הסחלב ומציגה המלצות מותאמות לפי כיוון החריגה (גבוה/נמוך) לכל חיישן. הפיצ'ר פונקציונאלי לחלוטין — הוא מעבד את הנתונים בזמן אמת ומציג למשתמש הנחיות טיפול ספציפיות לסחלב בלחיצת כפתור.

לאחר לחיצה על "Fetch readings" במסך ה-IoT Sensor Sampling, מתחת לטבלת הדגימות ולכרטיסי הסיכום (ממוצע/מינימום/מקסימום) מופיעות תיבות צהובות עם המלצות ספציפיות אם חריגה זוהתה, או תיבה ירוקה אם כל הנתונים תקינים.

הפיצ'ר ממוקם בתא 7 במחברת:

1. הגדרת הודעות - מילון ALERTS (שורות {...}) מגדיר 6 הודעות, אחת לכל חיישן וכיוון חריגה (high/low).
2. לוגיקה - בתוך הפונקציה build_sensor_html, הבלוק:

```
alert_set = set()
for v in vals:
    if v < t["min"]: alert_set.add(ALERTS[feed]["low"])
    elif v > t["max"]: alert_set.add(ALERTS[feed]["high"])
```

עובר על כל הדגימות ואוסף את ההתראות הרלוונטיות.

3. הרינדור - מיד אחרי ה-alert_set if מייצר תיבות HTML צהובות/ירוקות שמוחדרות ל-{alert_html} בתוצאה הסופית.